

Министерство образования Республики Беларусь

Учреждение образования
«Белорусский государственный университет информатики и
радиоэлектроники»

Кафедра ЭВМ

Лабораторная работа № 6
«Высокоуровневые библиотеки. SD-карта. Прямой доступ к памяти»
Вариант №9

Выполнил:
Студент группы 250501 Снитко Д.А.

Проверил:
ассистент каф. ЭВМ _____ Шеменков В.В.

Минск 2025

1. Цель работы

Цель: Изучить принципы организации прямого доступа к памяти на базе микроконтроллера MSP430F5529 и работы с SD-картой на основе экспериментальной платы MSP-EXP430F5529. Получить навыки комплексного использования периферийных устройств микроконтроллера MSP430F5529 и устройств экспериментальной платы MSP-EXP430F5529.

Задача: Написать программу организующую обмен с FLASH-памятью, SD-картой с применением прямого доступа к памяти и с использованием высокоуровневых библиотек.

2. Исходные данные

Для выполнения работы используется плата MSP-EXP430F5529 и интегрированная среда разработки Code Composer Studio.

Необходимо выполнить задание варианта № 3.

Измеряемый сигнал	Кнопка для чтения	Частота снятия данных, Гц	Размер буфера, байт	Режим отображения
PAD3	Ось Y акселерометра	40	100	Автоматически по заполнении буфера

3. Выполнение работы

Измеряем сигнал по PAD3, он записывается на SD-карту, по наклону переходит в режим чтения с SD-карты и отображает то, что было на нее записано.

3.1 Работа платы

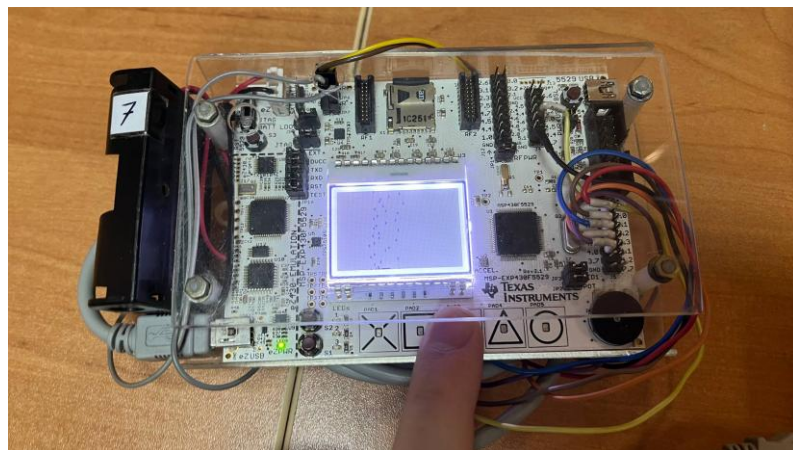


Рисунок 3.1.1 - Отображение значений в реальном времени

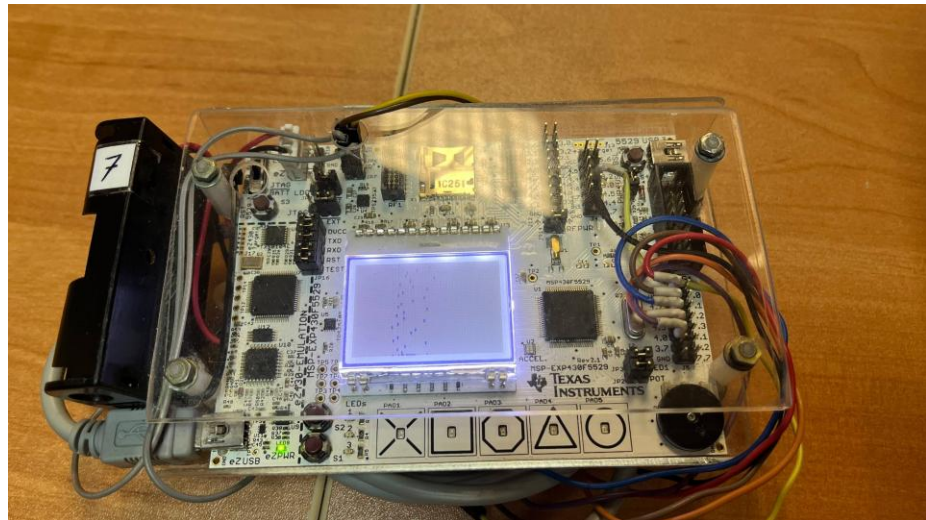


Рисунок 3.1.2 - Отображение сохраненного графика

3.2 Код программы

```
#include <msp430.h>
#include "HAL_Dogs102x6.h"
#include "HAL_Cma3000.h"
#include "HAL_Buttons.h"
#include "ff.h"
#include "structure.h"
#include "CTS_Layer.h"
#include <stdlib.h>
#include <math.h>

// имя файла для хранения буфера
#define FILE_NAME "data.bin"

//буфер
#define BUFFER_SIZE_BYTES 100
#define BUFFER_COUNT (BUFFER_SIZE_BYTES/2)
volatile uint16_t buffer[BUFFER_COUNT];
volatile uint8_t index = 0;
FATFS fs;
FIL file;
uint8_t file_draw = 0;
uint16_t tilt_y = 0;
uint8_t tilt_in_zone = 0;
uint8_t tilt_state = 0;
uint8_t prev_tilt_state = 0;
```

```

static int MAPPING_VALUES[] = { 1142, 571, 286, 143, 71, 36,
18 };
static unsigned char BITx[] = { BIT6, BIT5, BIT4, BIT3, BIT2,
BIT1, BIT0 };

#define TILT_ENTER_VALUE 550
#define TILT_EXIT_VALUE 520

uint16_t pad3_value = 0;

inline void readPad3();
uint16_t getFormattedValue(uint8_t index);
void drawBuffer();
void drawStored();
void storeBuffer();
inline void readTiltY();
inline void status_meas();
inline void status_file();

inline void readPad3()
{
    ADC12CTL0 = ADC12SHT0_8 | ADC12ON;
    ADC12CTL1 = ADC12SHP;
    ADC12MCTL0 = ADC12SREF_0 | ADC12INCH_3;
    P6SEL |= BIT3;
    ADC12CTL0 |= ADC12ENC;
    ADC12CTL0 |= ADC12SC;
    while (!(ADC12IFG & ADC12IFG0));
    pad3_value = ADC12MEM0;
}

inline void readTiltY()
{
    unsigned char b = Cma3000_readRegister(DOUTY);
    int isNeg = b & BIT7;
    int v = 0;
    int i;
    for (i = 0; i < 7; i++) {
        v += (isNeg ? ((BITx[i] & b) ? 0 : MAPPING_VALUES[i])
: ((BITx[i] & b) ? MAPPING_VALUES[i] : 0));
    }
    tilt_y = (uint16_t)abs(v);
}

```

```

inline void status_meas()
{
    Dogs102x6_stringDraw(0, 0, "meas",
DOGS102x6_DRAW_NORMAL);
}

inline void status_file()
{
    Dogs102x6_stringDraw(0, 0, "read",
DOGS102x6_DRAW_NORMAL);
}

uint16_t getFormattedValue(uint8_t index)
{
    uint16_t draw_value = (uint16_t)((float)DOGS102x6_Y_SIZE
* (float)buffer[index] / 4095.0f);
    return draw_value;
}

void drawBuffer()
{
    Dogs102x6_clearScreen();
    uint16_t i = 0;
    for (i = 0; i < BUFFER_COUNT; i++)
    {
        uint16_t x = (uint16_t)((uint32_t)i *
DOGS102x6_X_SIZE / BUFFER_COUNT);
        uint16_t draw_value = getFormattedValue(i);
        Dogs102x6_verticalLineDraw(x, x+1, draw_value,
DOGS102x6_DRAW_NORMAL);
    }
}

void drawStored()
{
    file_draw ^= 1;
    if (file_draw)
    {
        TA2CTL &= ~MC__UP;
        f_open(&file, FILE_NAME, FA_READ);
        f_read(&file, (void*) buffer, BUFFER_SIZE_BYTES,
NULL);

        f_close(&file);
        Dogs102x6_clearScreen();
    }
}

```

```

        status_file();
        drawBuffer();
    } else {
        Dogs102x6_clearScreen();
        status_meas();
    }
}

void storeBuffer()
{
    f_open(&file, FILE_NAME, FA_WRITE | FA_CREATE_ALWAYS);
    f_write(&file, (void*) buffer, BUFFER_SIZE_BYTES, NULL);
    f_close(&file);
}

uint16_t main(void)
{
    WDTCTL = WDTPW + WDTHOLD;

    Buttons_init(BUTTON_S1);
    Buttons_interruptEnable(BUTTON_ALL);

    Dogs102x6_init();
    Dogs102x6_backlightInit();
    Dogs102x6_setBacklight(255);
    Dogs102x6_clearScreen();
    status_meas();

    Cma3000_init();

    FRESULT res = f_mount(0, &fs);
    if (res == FR_NO_FILESYSTEM) {
        // f_mkfs(0, 0, 512);
    }

    DMACTL0 = DMA0TSEL_5;
    DMA0CTL = DMADT_0+DMAEN+DMAIE;
    DMA0SZ = 1;
    __data16_write_addr((unsigned short) &DMA0SA, (unsigned
long) &pad3_value);
    __data16_write_addr((unsigned short) &DMA0DA, (unsigned
long) &buffer[index]);

    readPad3();
}

```

```

readTiltY();
prev_tilt_state = (tilt_y >= TILT_ENTER_VALUE) ? 1 : 0;
tilt_state = prev_tilt_state;

TA2CCR0 = 800;
TA2CTL = TIMER_ACLK + TIMER_SOURCE_DIV_0;
TA2CTL |= (TACLK + MC__UP);

while (1)
{
    __bis_SR_register(LPM0_bits+GIE);

    readTiltY();
    {
        uint8_t new_state = prev_tilt_state;
        if (prev_tilt_state == 0 && tilt_y >=
TILT_ENTER_VALUE) new_state = 1;
        else if (prev_tilt_state == 1 && tilt_y <=
TILT_EXIT_VALUE) new_state = 0;

        if (!file_draw && prev_tilt_state == 0 &&
new_state == 1) {
            drawStored();
        } else if (file_draw && prev_tilt_state == 1 &&
new_state == 0) {
            drawStored();
        }
        prev_tilt_state = new_state;
        tilt_state = new_state;
    }
    if (!file_draw) {
        readPad3();
    }

    if(buttonsPressed & BUTTON_S1){
        drawStored();
        buttonsPressed &= ~BUTTON_S1;
    }

    if (!file_draw) {
        ++index;
        if (index == BUFFER_COUNT)
        {
            Dogs102x6_clearScreen();

```

```

        status_meas();
        drawBuffer();
        storeBuffer();
        index = 0;
    }
}

//обновляем адрес назначения DMA
if (!file_draw) {
    __data16_write_addr((unsigned short)
&DMA0DA, (unsigned long) &buffer[index]);
}

//запуск DMA
if (!file_draw) {
    DMA0CTL |= DMAEN;
}

//запуск Таймера 2
if (!file_draw) {
    TA2CTL |= (TACLRL + MC__UP);
}
}

#pragma vector=DMA_VECTOR
__interrupt void DMA_ISR(void)
{
    switch(__even_in_range(DMAIV,16))
    {
        //прерывание DMA0IFG
        case 2:
            TA2CTL &= ~MC__UP; //остановка таймера
            __bic_SR_register_on_exit(LPM0_bits); // выход из
LPM0

            break;
        default: break;
    }
}

```

4. Выводы

В ходе выполнения лабораторной работы удалось изучить принципы работы с SD картой и прямым доступом к памяти на основе микроконтроллера MSP430F5529.